

Design and Optimization of a JPEG-Based Image Compression Algorithm

전공: 항공전자정보

학번: 2019124018

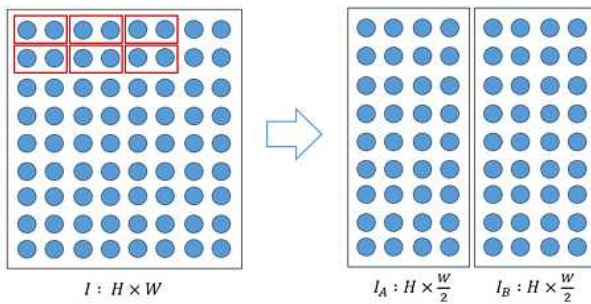
이름: 김동현

※ 공통으로 쓰이는 코드(ex. 이미지 시각화, MSE 함수 등)는 12p 부록에 정리되어 있습니다.

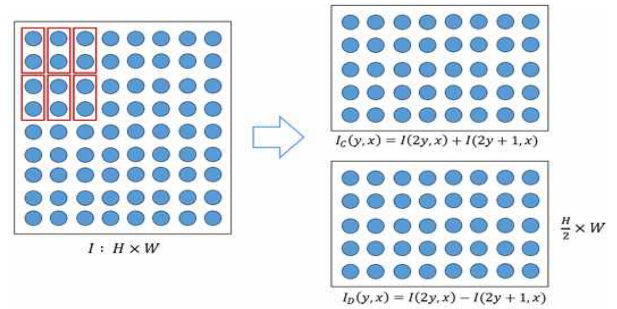
※ 해당 프로젝트는 (512×512) 크기의 동일한 이미지를 사용합니다.

1. 주어진 H×W 크기의 영상에 대해 아래 과정을 수행한다.

A. 영상 분리



[A_i 그림]



[A_ii 그림]

i. 위 그림과 같이 가로 방향의 인접한 두 픽셀의 합과 차를 각각 구하여, H×(W/2) 크기의 영상 2개를 만든다.

[접근] 다음 두 식 $I_A(y, x) = I(y, 2x) + I(y, 2x + 1)$, $I_B(y, x) = I(y, 2x) - I(y, 2x + 1)$ 을 이미지의 RGB채널에 각각 적용하여 두 개의 이미지를 만든다.

[핵심코드]

```
def process_horizontal(img):
    H, W, C = img.shape
    I_A = np.zeros((H, W // 2, C), dtype=np.int32)
    I_B = np.zeros((H, W // 2, C), dtype=np.int32)
    for c in range(C):
        for y in range(H):
            for x in range(0, W, 2):
                I_A[y, x // 2, c] = img[y, x, c] + img[y, x + 1, c]
                I_B[y, x // 2, c] = img[y, x, c] - img[y, x + 1, c]
    return I_A, I_B
```

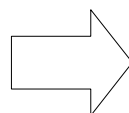
입력 이미지의 크기 추출
I_A 이미지 초기화
I_B 이미지 초기화
[접근] 식 적용
[접근] 식 적용

[코드설명] 반복문을 통하여 이미지 전반에 걸쳐 인접한 두 픽셀의 합과 차를 구한 후, I_A와 I_B에 각각 저장한다.

[실행결과] ※ 정상적인 출력을 위해 모든 픽셀 값을 8비트로 정규화하여 이미지를 출력하였습니다.



원본 이미지



I_A 이미지



I_B 이미지

ii. 유사한 방식으로 세로 방향의 인접한 두 픽셀의 합과 차를 각각 구하여, $(H/2) \times W$ 크기의 영상 2개를 만든다.

[접근] 다음 두 식 $I_C(y,x)=I(2y,x)+I(2y+1,x)$, $I_D(y,x)=I(2y,x)-I(2y+1,x)$ 을 이미지의 RGB채널에 각각 적용하여 두 개의 이미지를 만든다.

[핵심코드]

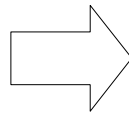
```
def process_vertical(img):
    H, W, C = img.shape
    I_C = np.zeros((H // 2, W, C), dtype=np.int32)
    I_D = np.zeros((H // 2, W, C), dtype=np.int32)
    for c in range(C):
        for y in range(0, H, 2):
            for x in range(W):
                I_C[y // 2, x, c] = img[y, x, c] + img[y + 1, x, c]
                I_D[y // 2, x, c] = img[y, x, c] - img[y + 1, x, c]
    return I_C, I_D
```

[코드설명] 합과 차를 구하는 방향을 제외하고는 [i 코드]와 동일하다.

[실행결과] ※ 정상적인 출력을 위해 모든 픽셀 값을 8비트로 정규화하여 이미지를 출력하였습니다.



원본 이미지



위: I_C 아래: I_D

B. 영상 복원

i. 문제 A- i 의 역과정을 구현하고, 원본 영상이 완벽하게 복원되는지 확인한다.

[접근] I_A 이미지와 I_B 이미지의 합과 차를 이용하여 원본 영상으로 복원한다.

[핵심코드]

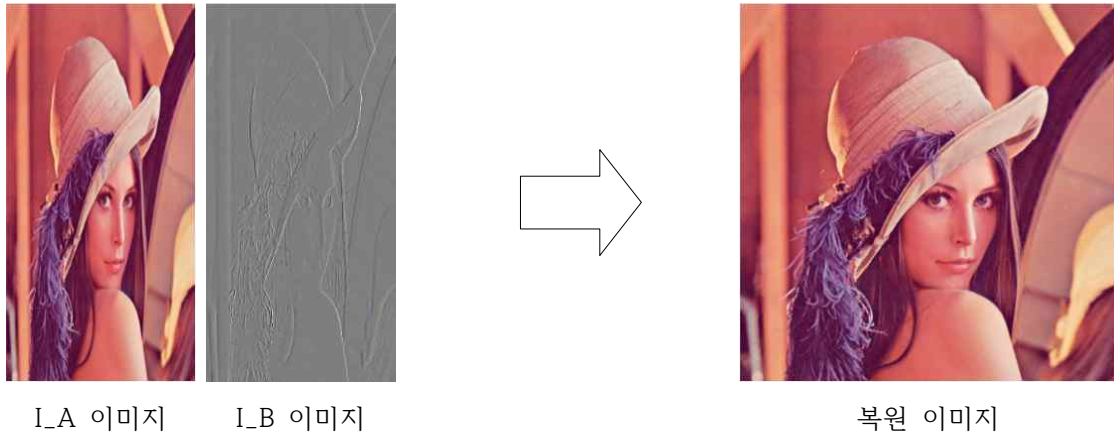
```
def restore_horizontal(I_A, I_B):
    H, W_half, C = I_A.shape
    W = W_half * 2
    restored = np.zeros((H, W, C), dtype=np.int32)
    for c in range(C):
        for y in range(H):
            for x in range(W_half):
                restored[y, 2 * x, c] = (I_A[y, x, c] + I_B[y, x, c]) // 2 # 0과 짝수 열 픽셀 값 복원
                restored[y, 2 * x + 1, c] = (I_A[y, x, c] - I_B[y, x, c]) // 2 # 홀수 열 픽셀 값 복원
    return restored
```

[코드설명] 홀수 열의 픽셀 값은 $(I_A(y,x)-I_B(y,x))/2$ 로 복원하며 짝수 및 첫 열 픽셀 값은 $(I_A(y,x)+I_B(y,x))/2$ 로 연

산한다. 반복문을 통해 전체 이미지를 복원할 수 있도록 한다.

[실행결과] 원본이미지와 복원이미지의 MSE는 0으로 계산된다. 따라서 완벽하게 복원되었음을 알 수 있다.

※ 정상적인 출력을 위해 모든 픽셀 값을 8비트로 정규화하여 이미지를 출력하였습니다.



ii. 문제 A-ii의 역과정을 구현하고, 원본 영상이 완벽하게 복원되는지 확인한다.

[접근] I_C 이미지와 I_D 이미지의 합과 차를 이용하여 원본 영상으로 복원한다.

[핵심코드]

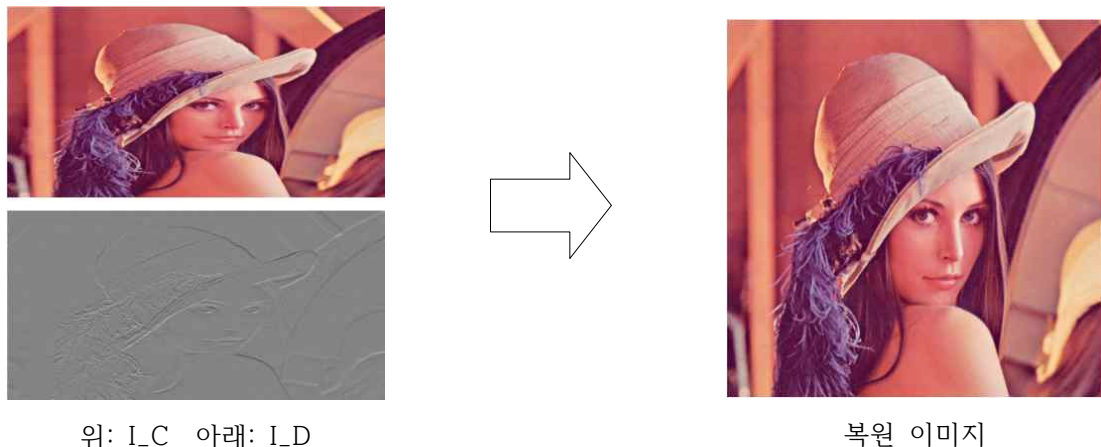
```
def restore_vertical(I_C, I_D):
    H_half, W, C = I_C.shape
    H = H_half * 2
    restored = np.zeros((H, W, C), dtype=np.int32)

    for c in range(C):
        for x in range(W):
            for y in range(H_half):
                restored[2 * y, x, c] = (I_C[y, x, c] + I_D[y, x, c]) // 2
                restored[2 * y + 1, x, c] = (I_C[y, x, c] - I_D[y, x, c]) // 2
    return restored
```

[코드설명] 홀수 행의 픽셀 값은 $(I_C(y,x) - I_D(y,x))/2$ 로 복원하며 짝수 및 첫 행의 픽셀 값은 $(I_C(y,x) + I_D(y,x))/2$ 로 연산한다. 반복문을 통해 전체 이미지를 복원할 수 있도록 한다.

[실행결과] 원본이미지와 복원이미지의 MSE는 0으로 계산된다. 따라서 완벽하게 복원되었음을 알 수 있다.

※ 정상적인 출력을 위해 모든 픽셀 값을 8비트로 정규화하여 이미지를 출력하였습니다.



C. 문제 A의 영상 분리 과정을 연속적으로 적용한다. A- i 와 A- ii 의 과정을 각각 3번 연속으로 적용하게 되면 $(H/8) \times (W/8)$ 크기의 영상이 64개 생기게 된다. 이때, 적용 순서에 따라 결과가 어떻게 변하는지 관찰한다. 또한, 역과정을 적용하여 원영상을 복원한다.

[접근] 문제A에서 작성한 함수 2개를 적절히 사용하여 64개의 영상을 만든다. 또한 적용 순서를 달리하여 결과를 비교해본다. 문제B에서 작성한 함수 2개를 적절히 사용하여 64개의 이미지를 원본 이미지로 복원한다. 마찬가지로 복원 순서를 달리하여 결과를 비교해본다.

[핵심코드]

[방식1_분할] A- i 3번 적용 후 A- ii 3번 적용

[방식2_분할] A- ii 3번 적용 후 A- i 3번 적용

<pre> # 가로 방향 I_A1, I_B1 = process_horizontal(image_rgb) I_A2_A, I_A2_B = process_horizontal(I_A1) I_B2_A, I_B2_B = process_horizontal(I_B1) # 가로 방향 3단계 I_A3_AA, I_A3_AB = process_horizontal(I_A2_A) I_A3_BA, I_A3_BB = process_horizontal(I_A2_B) I_B3_AA, I_B3_AB = process_horizontal(I_B2_A) I_B3_BA, I_B3_BB = process_horizontal(I_B2_B) # 가로 방향 3단계 결과 리스트 horizontal_images = [I_A3_AA, I_A3_AB, I_A3_BA, I_A3_BB, I_B3_AA, I_B3_AB, I_B3_BA, I_B3_BB] # 세로 방향 3단계 진행 vertical_final_results = [] for img in horizontal_images: # 1단계 V1, V2 = process_vertical(img) # 2단계 V3, V4 = process_vertical(V1) V5, V6 = process_vertical(V2) # 3단계 V7, V8 = process_vertical(V3) V9, V10 = process_vertical(V4) V11, V12 = process_vertical(V5) V13, V14 = process_vertical(V6) vertical_final_results.extend([V7, V8, V9, V10, V11, V12, V13, V14]) </pre>	<pre> # 세로 방향 I_A1, I_B1 = process_vertical(image_rgb) I_A2_A, I_A2_B = process_vertical(I_A1) I_B2_A, I_B2_B = process_vertical(I_B1) # 세로 방향 3단계 I_A3_AA, I_A3_AB = process_vertical(I_A2_A) I_A3_BA, I_A3_BB = process_vertical(I_A2_B) I_B3_AA, I_B3_AB = process_vertical(I_B2_A) I_B3_BA, I_B3_BB = process_vertical(I_B2_B) # 세로 방향 3단계 결과 리스트 vertical_images = [I_A3_AA, I_A3_AB, I_A3_BA, I_A3_BB, I_B3_AA, I_B3_AB, I_B3_BA, I_B3_BB] # 가로 방향 3단계 진행 horizontal_final_results = [] for img in vertical_images: # 1단계 V1, V2 = process_horizontal(img) # 2단계 V3, V4 = process_horizontal(V1) V5, V6 = process_horizontal(V2) # 3단계 V7, V8 = process_horizontal(V3) V9, V10 = process_horizontal(V4) V11, V12 = process_horizontal(V5) V13, V14 = process_horizontal(V6) horizontal_final_results.extend([V7, V8, V9, V10, V11, V12, V13, V14]) </pre>
---	---

[코드설명] 생성된 개별 이미지들을 다음 함수의 입력 값으로 하는 다단계 방식으로 코드를 작성하였다. [방식1]의 경우 A- i 3번 적용하면 총 8개의 이미지가 생성된다. 각 이미지들을 A- ii 에 한 번 거치게 되면 16개의 이미지가 생성된다. 이를 총 3번 반복하면 8, 16, 32, 64 순으로 이미지가 생성되며 최종적으로 64개의 이미지를 얻는다. [방식2]의 경우도 함수 사용 순서만 다를 뿐 전반적인 알고리즘은 동일하다.

[방식1_복원]

```
restored_images = []

# 세로 방향 결과를 그룹별로 처리
for i in range(0, len(vertical_final_results), 8):
    group = vertical_final_results[i:i + 8]
    V3_re = restore_vertical(group[0], group[1])
    V4_re = restore_vertical(group[2], group[3])
    V5_re = restore_vertical(group[4], group[5])
    V6_re = restore_vertical(group[6], group[7])

    V1_re = restore_vertical(V3_re, V4_re)
    V2_re = restore_vertical(V5_re, V6_re)

    restored_group = restore_vertical(V1_re, V2_re)
    restored_images.append(restored_group)

# 가로 방향 복원
final_restored_image = restore_horizontal(
    restore_horizontal(
        restore_horizontal(restored_images[0],
restored_images[1]),
        restore_horizontal(restored_images[2],
restored_images[3])
    ),
    restore_horizontal(
        restore_horizontal(restored_images[4],
restored_images[5]),
        restore_horizontal(restored_images[6],
restored_images[7])
    )
)
```

[방식2_복원]

```
restored_images = []

# 가로 방향 결과를 그룹별로 처리
for i in range(0, len(horizontal_final_results), 8):
    group = horizontal_final_results[i:i + 8]
    H3_re = restore_horizontal(group[0], group[1])
    H4_re = restore_horizontal(group[2], group[3])
    H5_re = restore_horizontal(group[4], group[5])
    H6_re = restore_horizontal(group[6], group[7])

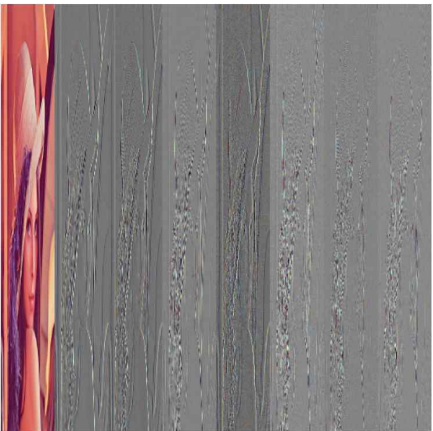
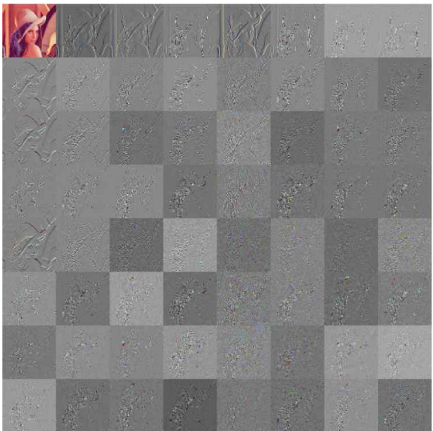
    H1_re = restore_horizontal(H3_re, H4_re)
    H2_re = restore_horizontal(H5_re, H6_re)

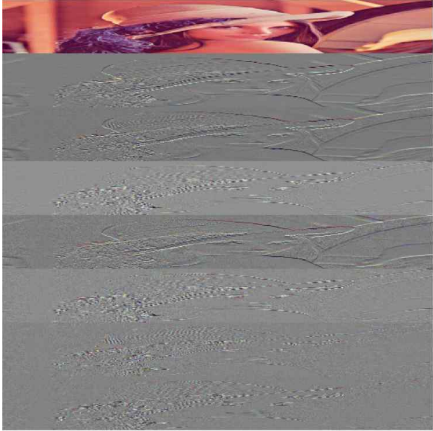
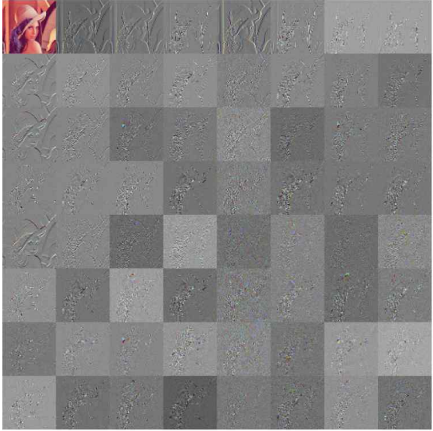
    restored_group = restore_horizontal(H1_re, H2_re)
    restored_images.append(restored_group)

# 세로 방향 복원
final_restored_image = restore_vertical(
    restore_vertical(
        restore_vertical(restored_images[0],
restored_images[1]),
        restore_vertical(restored_images[2],
restored_images[3])
    ),
    restore_vertical(
        restore_vertical(restored_images[4],
restored_images[5]),
        restore_vertical(restored_images[6],
restored_images[7])
    )
)
```

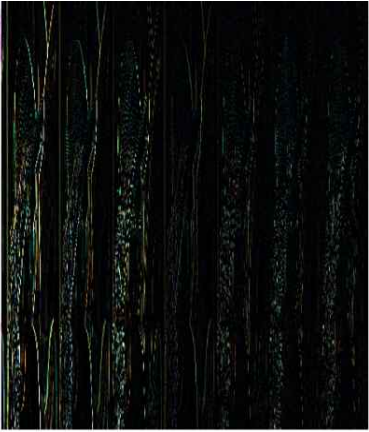
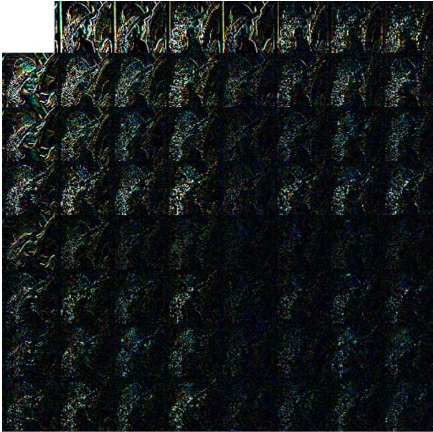
[코드설명] 64개로 분할된 이미지를 복원하는 코드이다. [방식1_복원]의 경우 [방식1] 역순으로 코드를 작성한다. vertical_final_results 리스트에는 8개의 그룹이 있으며, 각 그룹에는 (3번의 A-ii 연산이 된) 8개의 이미지가 있다. 8개의 이미지를 2개씩 묶어서 세로 방향으로 복원 함수 restore_vertical에 입력하면 4개의 이미지로 복원된다. 각 그룹에 이러한 연산을 3번 적용하면 결과적으로 총 (A-i 3번 적용된) 8개의 이미지로 복원된다. 이렇게 복원된 이미지들을 2개씩 짝지어서 가로 방향 복원 함수 restore_horizontal에 입력하면 최종적으로 원본이미지로 복원된다. [방식2_복원]도 동일하다.

[실행결과] ※ 정상적인 출력을 위해 이미지를 정규화하여 출력하였다.

	원본 이미지	중간 단계: A-i 3번 적용	최종: A-ii 3번 적용
방식1 분할			

	원본 이미지	중간 단계: A- ii 3번 적용	최종: A- i 3번 적용
방식2 분할			

	원본 이미지	방식1_복원, MSE: 0.00	방식2_복원, MSE: 0.00
방식별 복원 이미지 비교			

	A- i 만 3번 적용	A- ii 만 3번 적용	64개로 분할된 이미지
정규화를 하지 않은 이미지			

각 연산 과정을 거치면서 픽셀 값이 0~255 범위를 벗어나게 되는데, 이럴 경우 정규화를 해주지 않으면 위와 같은 정규화를 하지 않은 이미지가 출력된다.

[고찰] 분할 과정에서 A- i , A- ii 적용 순서를 바꿔도 동일한 결과가 출력된다. 이는 두 방식 A- i , A- ii가 선형시스템이기 때문이다. 즉, 적용 순서가 결과에 영향을 미치지 않는다. 복원도 마찬가지로 두 가지 과정이 각각 선형 시스템이기 때문에 원본이미지로 완벽하게 복원된다. 이는 원본이미지와 복원이미지의 MSE 값이 0이 출력됨을 통해 확인할 수 있다.

2. 문제 1의 변환에서 Quantization을 적용하여 영상 압축을 진행한다.

A. 각각의 64개의 영상에 대하여 각각 동일한 quantization parameter 값을 적용하여, 각각의 영상에서의 rate와 distortion을 구한다.

[접근] 문제 A에서 생성한 64개의 이미지 각각 '양자화 - zigzag 재배열 - unary coding' 하고 복원하여 각각의 영상에서의 rate와 distortion을 구한다. 압축과 복원에서의 각 단계는 함수를 정의하여 사용한다.

[핵심코드]

압축에 필요한 함수

```
# 양자화 수행
def quantize(image, quant_table, QP):
    quant_table = quant_table * QP
    return np.round(image / quant_table).astype(int)

# 양자화 테이블 64x64
luminance_quant_table = np.ones((64, 64))
chrominance_quant_table = np.ones((64, 64))

# Zig-Zag 재배열
def zigzag(image):
    rows, cols = image.shape
    result = []
    for d in range(rows + cols - 1):
        if d % 2 == 1: # 홀수 대각선: 아래에서 위로
            for i in range(max(0, d - cols + 1), min(d + 1, rows)):
                result.append(image[i, d - i])
        else: # 짝수 대각선: 위에서 아래로
            for i in range(max(0, d - rows + 1), min(d + 1, cols)):
                result.append(image[d - i, i])
    return np.array(result)

# Unary 코딩
def unary_encode(zigzag_result):
    unary_encoded = []
    for value in zigzag_result:
        if value >= 0:
            unary_encoded.append('1' * value + '0')
        else:
            unary_encoded.append('1' * abs(value) + '0' + '-')
    return unary_encoded

# Y 이미지 양자화, Zig-Zag 재배열, Unary
def image_compress(image, quant_table, QP):
    image = quantize(image, quant_table, QP) # 양자화
    zigzag_result = zigzag(image) # Zig-Zag 재배열
    unary_result = unary_encode(zigzag_result) # unary 코딩
    return unary_result # 리스트 형태
```

복원에 필요한 함수

```
# 양자화 역변환
def dequantize(quantized_image, quant_table, QP):
    adjusted_quant_table = quant_table * QP
    return (quantized_image * adjusted_quant_table).astype(float)

# Zig-Zag 역재배열
def inverse_zigzag(arr):
    result = np.zeros((64, 64), dtype=float)
    index = 0
    for d in range(64 + 64 - 1):
        if d % 2 == 1: # 홀수 대각선: 아래에서 위로
            for i in range(max(0, d - 64 + 1), min(d + 1, 64)):
                result[i, d - i] = arr[index]
                index += 1
        else: # 짝수 대각선: 위에서 아래로
            for i in range(max(0, d - 64 + 1), min(d + 1, 64)):
                result[d - i, i] = arr[index]
                index += 1
    return result
```



```

# Unary 디코딩
def unary_decode(compressed_image):
    decoded = []
    for code in compressed_image:
        if code.endswith('-'):
            value = -len(code[:-1].rstrip('0'))
        else:
            value = len(code.rstrip('0'))
        decoded.append(value)
    return np.array(decoded)

# 압축된 결과로부터 원본 이미지 복원
def image_decompress(compressed_image, quant_table, QP):
    zigzag_result = unary_decode(compressed_image)
    dezigzagged = inverse_zigzag(zigzag_result)
    restored_image = dequantize(dezigzagged, quant_table, QP)
    return restored_image

```

Unary 디코딩
 # Zig-Zag 역재배열
 # 양자화 역변환

압축 및 복원 실행

```

for i, image in enumerate(image_list):
    QP = 100
    compressed_image_Y = image_compress(image[:, :, 0], luminance_quant_table, QP)
    compressed_image_U = image_compress(image[:, :, 1], chrominance_quant_table, QP)
    compressed_image_V = image_compress(image[:, :, 2], chrominance_quant_table, QP)

    decompressed_image_Y = image_decompress(compressed_image_Y, luminance_quant_table, QP)
    decompressed_image_U = image_decompress(compressed_image_U, chrominance_quant_table, QP)
    decompressed_image_V = image_decompress(compressed_image_V, chrominance_quant_table, QP)

    final_image = np.stack((decompressed_image_Y, decompressed_image_U, decompressed_image_V),
axis=2)

```

[코드설명] 압축에 필요한 과정들을 함수로 정의하였고 실질적으로 압축을 진행하는 함수 `image_compress(image, quant_table, QP)` 에 순서대로 동작하도록 배치하였다. 복원 과정도 압축 과정과 마찬가지로 각 과정을 함수로 정의하였고 `image_decompress(compressed_image, quant_table, QP)` 에 순서대로 동작하도록 배치하였다. 실제 이미지에 적용할 때는 Y,U,V 채널을 나눠서 각각 압축 및 복원을 하고 마지막에 `np.stack`을 이용하여 하나로 묶어주었다.

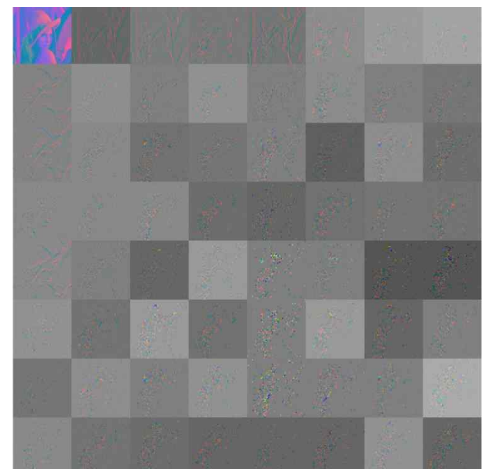
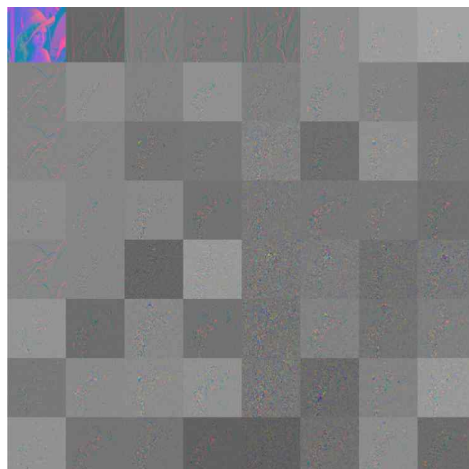
[실행결과] QP(quantization parameter) 값은 luminance와 chrominance 모두 100로 설정하였다.

원본 YUV 이미지

복원 YUV 이미지

[그림] image number

1	2	3	4	5	6	7	8
9	10	11	12	13	14	15	16
17	18	19	20	21	22	23	24
25	26	27	28	29	30	31	32
33	34	35	36	37	38	39	40
41	42	43	44	45	46	47	48
49	50	51	52	53	54	55	56
57	58	59	60	61	62	63	64



위 [그림] image number와 아래 표를 이용하여 각각의 이미지에서의 rate와 distortion을 확인할 수 있다.

빨강: 최댓값 파랑: 최솟값

이미지 번호	1	2	3	4	5	6	7	8
rate	271.88	8.88	6.03	5.67	4.50	4.25	3.96	4.00
distortion	833.14	794.45	737.19	691.91	644.09	615.88	563.98	581.67
이미지 번호	9	10	11	12	13	14	15	16
rate	6.42	5.28	4.18	4.45	3.50	3.70	3.66	3.72
distortion	742.69	649.41	568.29	572.55	496.49	522.38	508.00	509.69
이미지 번호	17	18	19	20	21	22	23	24
rate	4.66	4.08	3.67	3.79	3.24	3.35	3.46	3.44
distortion	636.93	546.94	495.04	505.68	405.29	441.13	443.63	443.63
이미지 번호	25	26	27	28	29	30	31	32
rate	4.36	4.25	3.77	4.01	3.29	3.47	3.53	3.54
distortion	589.24	544.98	505.05	507.05	430.57	475.91	470.51	475.41
이미지 번호	33	34	35	36	37	38	39	40
rate	3.73	3.43	3.19	3.24	3.08	3.12	3.12	3.14
distortion	489.40	411.25	348.33	359.67	294.65	317.44	323.10	324.93
이미지 번호	41	42	43	44	45	46	47	48
rate	3.58	3.52	3.34	3.41	3.11	3.18	3.23	3.21
distortion	455.98	420.88	383.38	395.73	328.21	352.80	363.42	359.73
이미지 번호	49	50	51	52	53	54	55	56
rate	3.46	3.46	3.42	3.44	3.13	3.20	3.35	3.30
distortion	468.12	424.40	427.08	416.28	357.47	388.76	405.71	403.77
이미지 번호	57	58	59	60	61	62	63	64
rate	3.48	3.39	3.39	3.46	3.13	3.20	3.30	3.25
distortion	458.42	425.86	413.98	414.17	346.38	377.64	378.49	389.42
Full image rate	7.97							
Full image distortion	474.53							

B. 다양한 quantization parameter를 적용하여 R-D curve를 얻고, 이 방법과 기존 JPEG 방법과의 성능을 비교분석한다.

[접근] 위 문제에서 QP 값을 변경해가면서 rate와 distortion을 구하고 R-D curve를 그린다. 또한 JPEG 방법을 [블록화-DCT-양자화-zigzag 재배열-Unary coding] 순으로 구현하여 R-D curve를 비교해본다. 문제2_A와 동일한 코드를 사용하고, JPEG 방법은 기존 방법에 8×8 블록화 함수, DCT 함수와 양자화 Table을 추가하여 사용한다. QP=[139, 270, 400, 700, 1600, 2600] 으로 한다.

[추가코드]

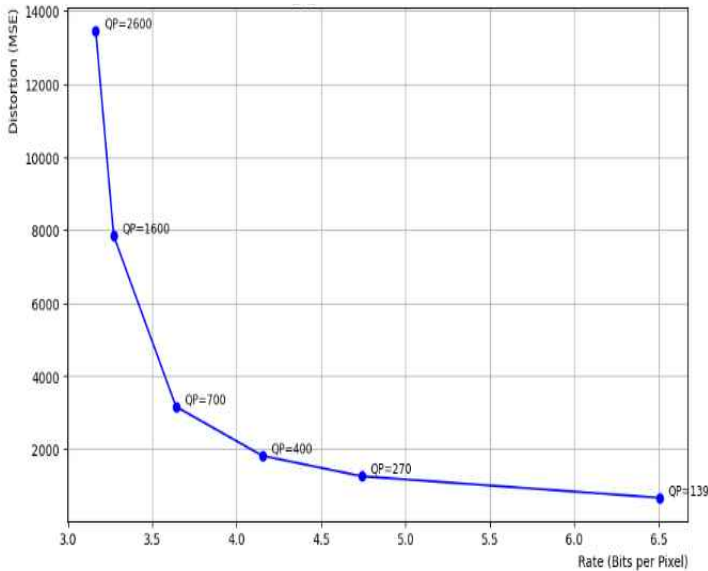
```
# 8*8 크기로 분할
def split_image_into_blocks(image_yuv, block_size):
    blocks = []
    height, width, channel = image_yuv.shape
    for y in range(0, height, block_size):
        for x in range(0, width, block_size):
            block = image_yuv[y:y + block_size, x:x + block_size, :]
            blocks.append(block)

    return blocks

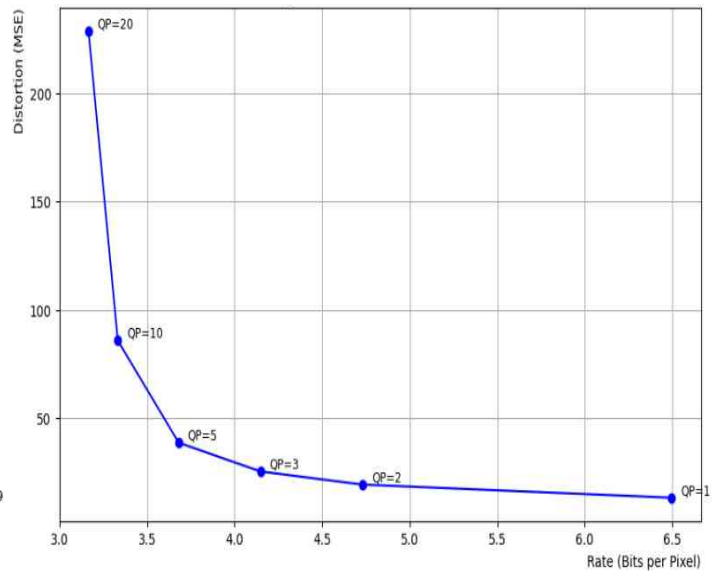
# 2D DCT 적용
def apply_2d_dct(image):
    return dct(dct(image.T, norm='ortho').T, norm='ortho')
```

[실행결과]

구현한 모델의 R-D curve



JPEG 압축의 R-D curve



QP 값을 조정하여 압축 모델의 Rate를 JPEG의 Rate 범위와 동일하게 해주었다. 따로 수치를 작성하지 않고 그래프만으로도 같은 Rate일 때 Distortion의 차이가 상당함을 알 수 있다. 따라서 현재 압축 모델은 JPEG보다 성능이 떨어진다.

	rate = 6.5	rate = 4.7	rate = 4.1	rate = 3.6	rate = 3.3	rate = 3.1
현재 모델 왜곡	666.78	1256.93	1818.40	3162.25	7843.11	13471.32
JPEG 모델 왜곡	13.03	4.73	4.15	3.68	3.33	3.17

C. 위 과정에서 64개의 (H/8)×(W/8) 크기의 영상마다의 최적의 quantization parameter를 적용한다면 압축성능을 향상시킬 수 있다. 우리만의 방법을 적용하여 최적의 quantization parameter 값을 찾는 방법을 제시한다.

[접근] 1. 최적의 QP를 찾기 위한 비용함수를 다음과 같이 설정한다.

$$\text{cost} = a \cdot \text{MSE} + b \cdot \text{Rate} \quad (a: \text{MSE의 중요도 } b: \text{Rate의 중요도})$$

2. 각 이미지의 각 채널별로 QP 값을 1부터 999까지 변경하면서 압축/복원을 반복한다.
3. 압축 복원을 반복하면서 계산된 cost 값 중 최소가 되는 QP 값을 저장한다.
4. 위 과정을 반복하여 각 영상 및 채널마다 cost가 최소가 되는 QP 값을 구한다.
5. 구한 QP 값들에 scaling을 하여 사용자가 원하는 최적의 QP 값들을 얻는다.

[핵심코드]

```
alpha = 0.05 # MSE 중요도
beta = 0.95 # Rate 중요도
for j in range(len(image_list)):
    min_cost = float('inf'); optimal_qp = None

    for i, QP in enumerate(range(1, 1000, 1)):
        compressed_image_Y = image_compress(image_list[j][:,:,0], luminance_quant_table, QP)
        decompressed_image_Y = image_decompress(compressed_image_Y, luminance_quant_table, QP)

        mse_value = calculate_mse(image_list[j][:,:,0], decompressed_image_Y)
        total_bits = sum(len(code) for code in compressed_image_Y)
        rate = total_bits / 4096 # 64*64
        cost = alpha * mse_value + beta * rate # 비용 함수 계산

    if cost < min_cost:
        min_cost = cost
        optimal_qp = QP
```

[실행결과]

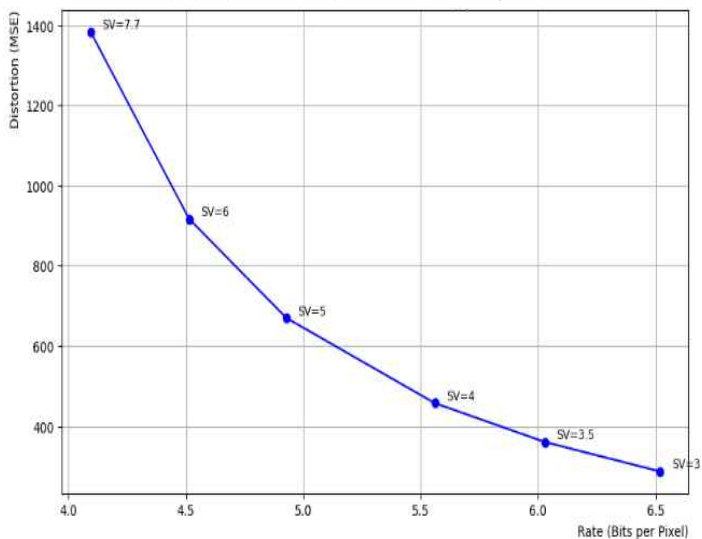
각 이미지별 최적의 QP_scale 3_(luminance channel)

image1	image2	image3	image4	image5	image6	image7	image8
285	105	78	81	63	60	60	60
image9	image10	image11	image12	image13	image14	image15	image16
84	78	57	63	48	54	51	57
image17	image18	image19	image20	image21	image22	image23	image24
66	63	51	57	45	48	48	48
image25	image26	image27	image28	image29	image30	image31	image32
63	63	57	60	45	48	51	54
image33	image34	image35	image36	image37	image38	image39	image40
54	48	42	45	36	42	39	42
image41	image42	image43	image44	image45	image46	image47	image48
51	51	42	45	42	42	42	42
image49	image50	image51	image52	image53	image54	image55	image56
48	48	45	48	39	42	48	48
image57	image58	image59	image60	image61	image62	image63	image64
51	51	48	51	42	45	45	42

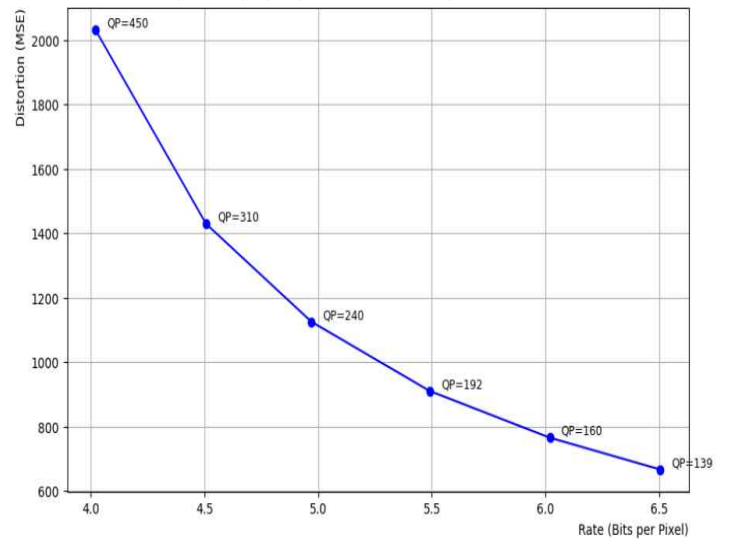
각 이미지별 최적의 QP_scale 3_(chrominance channel)

image1	image2	image3	image4	image5	image6	image7	image8
297	63	51	48	42	42	42	42
image9	image10	image11	image12	image13	image14	image15	image16
54	48	42	42	42	39	39	42
image17	image18	image19	image20	image21	image22	image23	image24
45	42	39	39	36	36	36	36
image25	image26	image27	image28	image29	image30	image31	image32
42	45	39	42	36	39	39	36
image33	image34	image35	image36	image37	image38	image39	image40
36	36	36	36	36	36	36	36
image41	image42	image43	image44	image45	image46	image47	image48
36	36	36	36	36	36	36	36
image49	image50	image51	image52	image53	image54	image55	image56
36	36	36	36	36	36	36	36
image57	image58	image59	image60	image61	image62	image63	image64
36	36	36	36	36	36	36	36

[접근] 4번에서 구한 QP의 scale을 조정한 R-D curve



[문제B] 에서 구현한 모델의 R-D curve



	rate = 6.5	rate = 6.0	rate = 5.5	rate = 5.0	rate = 4.5	rate = 4.0
최적화 모델의 왜곡	287.15	360.30	457.52	669.86	917.13	1384.14
[문제B] 모델의 왜곡	666.78	765.65	910.40	1125.23	1429.58	2032.88

<부록>

이 페이지에는 프로젝트에서 공통으로 사용되는 코드에 대한 설명이 작성되어있습니다.

```
import numpy as np          # 수치 연산을 위한 라이브러리
import matplotlib.pyplot as plt  # 데이터 시각화를 위한 라이브러리
import cv2                  # OpenCV 라이브러리
from scipy.fftpack import dct, idct  # DCT, IDCT를 scipy.fftpack에서 가져옴
from google.colab import drive      # Google Drive를 연결하는 모듈
drive.mount('/content/drive')       # 파일 시스템 연결
```

```
def calculate_mse(original, restored):    # MSE 계산 함수
    return np.mean((original - restored) ** 2)
```

이미지 로드 및 변환 (RGB 이미지)

```
image = cv2.imread('/content/drive/My Drive/Colab/lena.bmp', cv2.IMREAD_COLOR)
image_rgb = cv2.cvtColor(image, cv2.COLOR_BGR2RGB).astype(np.float32)
image_gray = cv2.cvtColor(image, cv2.COLOR_BGR2GRAY).astype(np.int32)
```

이미지 시각화에 사용되는 코드

```
plt.figure(figsize=(6, 6))
for i, img in enumerate(normalized_images):
    plt.subplot(1, 8, i + 1)
    plt.imshow(img)
    plt.axis('off')
```